



Strategi Keamanan Perangkat Lunak

Prinsip *Secure Design*

Program Studi: Teknik Informatika - Unjani

Dosen: Asep Id Hadiana, Ph.D.



Pengantar: Prinsip Desain Aman

Definisi

Prinsip desain aman adalah kumpulan pedoman fundamental yang digunakan untuk membangun sistem perangkat lunak yang resisten terhadap ancaman keamanan. Prinsip-prinsip ini memberikan landasan teoritis untuk mengembangkan aplikasi yang dapat melindungi data, menjaga integritas sistem, dan memastikan ketersediaan layanan.



Konsep ini pertama kali dirumuskan oleh **Saltzer dan Schroeder** pada tahun **1975** dalam paper klasik "The Protection of Information in Computer Systems" dan tetap relevan hingga era komputasi modern saat ini.



Tujuan Pembelajaran

Memahami Konsep

Menguasai sembilan prinsip desain aman dan filosofi di balik setiap prinsip tersebut

Penerapan Praktis

Mampu mengimplementasikan prinsip-prinsip dalam desain sistem perangkat lunak nyata

Analisis Kasus

Mengenali dan menganalisis penerapan prinsip desain aman dalam sistem dunia nyata

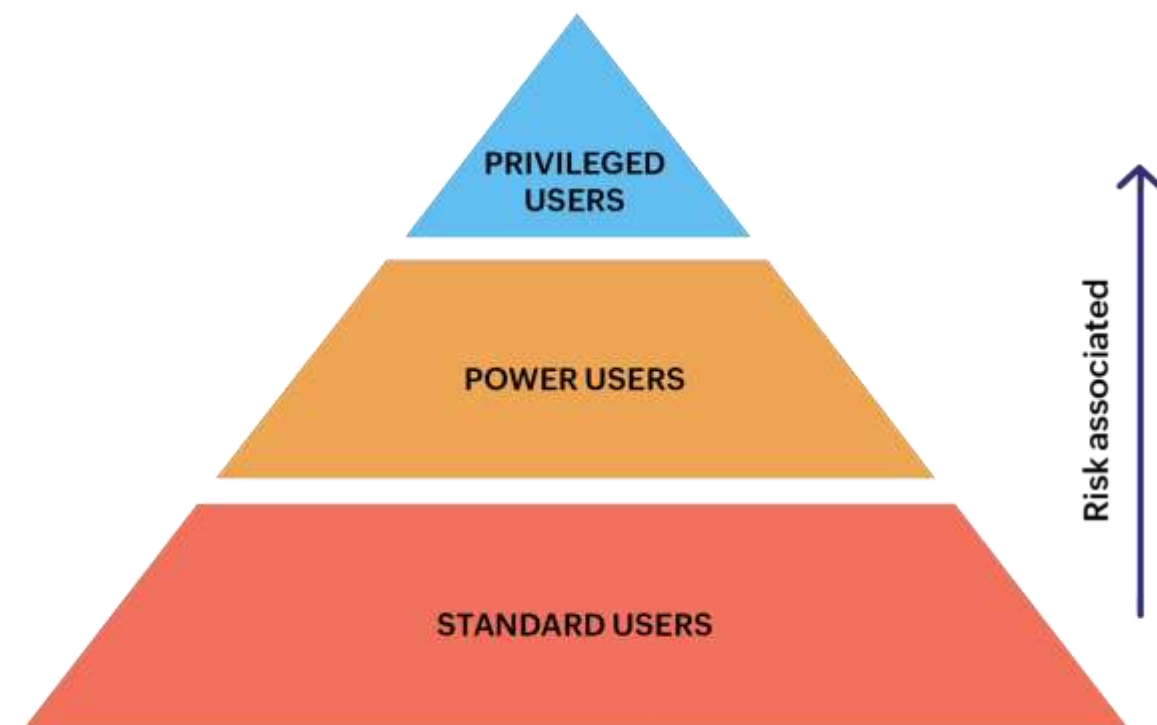
Prinsip 1: Least Privilege

Konsep Hak Akses Minimum

Setiap subjek (pengguna, proses, atau program) harus diberikan **hak akses minimum** yang diperlukan untuk menjalankan fungsinya. Tidak lebih, tidak kurang.

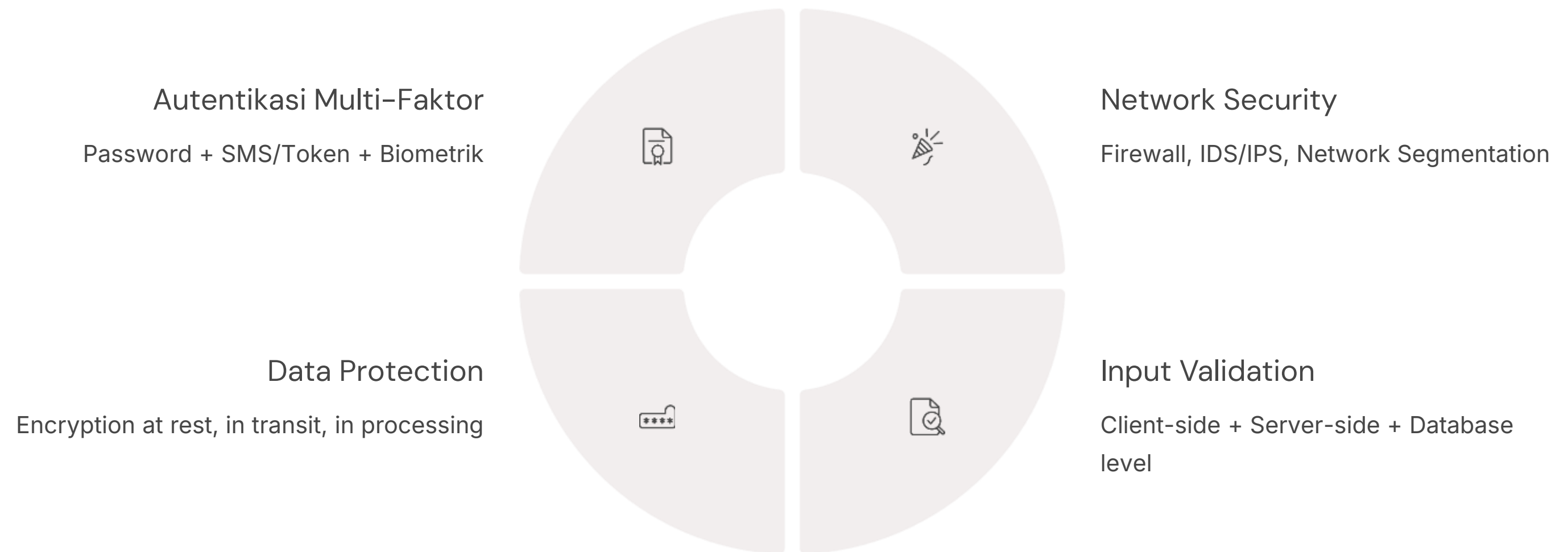
Contoh Implementasi:

- User aplikasi hanya dapat mengakses data miliknya
- Database connection dengan privilege terbatas
- Sistem operasi: user biasa vs administrator
- API endpoint dengan role-based access control



Prinsip 2: Defense in Depth

Perlindungan Berlapis



Jika satu lapisan keamanan gagal, lapisan lain masih dapat melindungi sistem dari ancaman.

Prinsip 3: Fail-Safe Defaults

Sistem Harus Gagal dalam Keadaan Aman

Ketika terjadi kegagalan atau error, sistem harus default ke kondisi yang paling aman, bukan yang paling permisif.

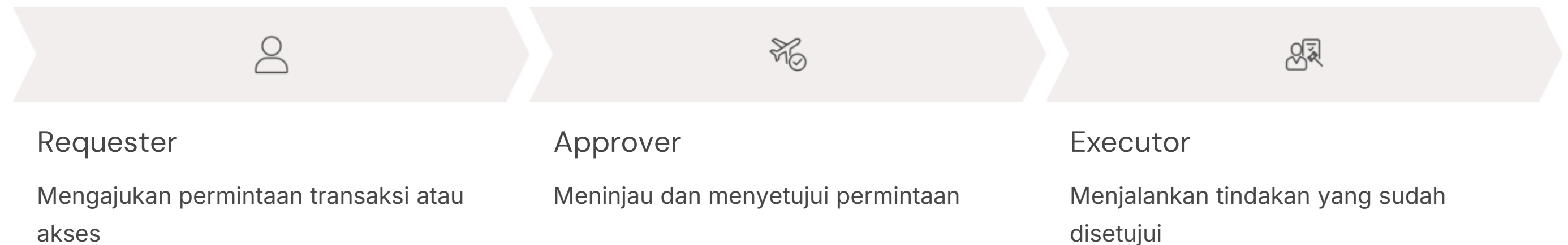
Contoh Penerapan:

- **Error Handling:** Tampilkan pesan generik, jangan expose detail sistem
- **Default Deny Policy:** Akses ditolak kecuali explicitly diizinkan
- **Firewall Rules:** Block all, allow specific
- **Session Timeout:** Otomatis logout saat idle



Prinsip 4: Separation of Duties

Pembagian Peran untuk Mencegah Penyalahgunaan



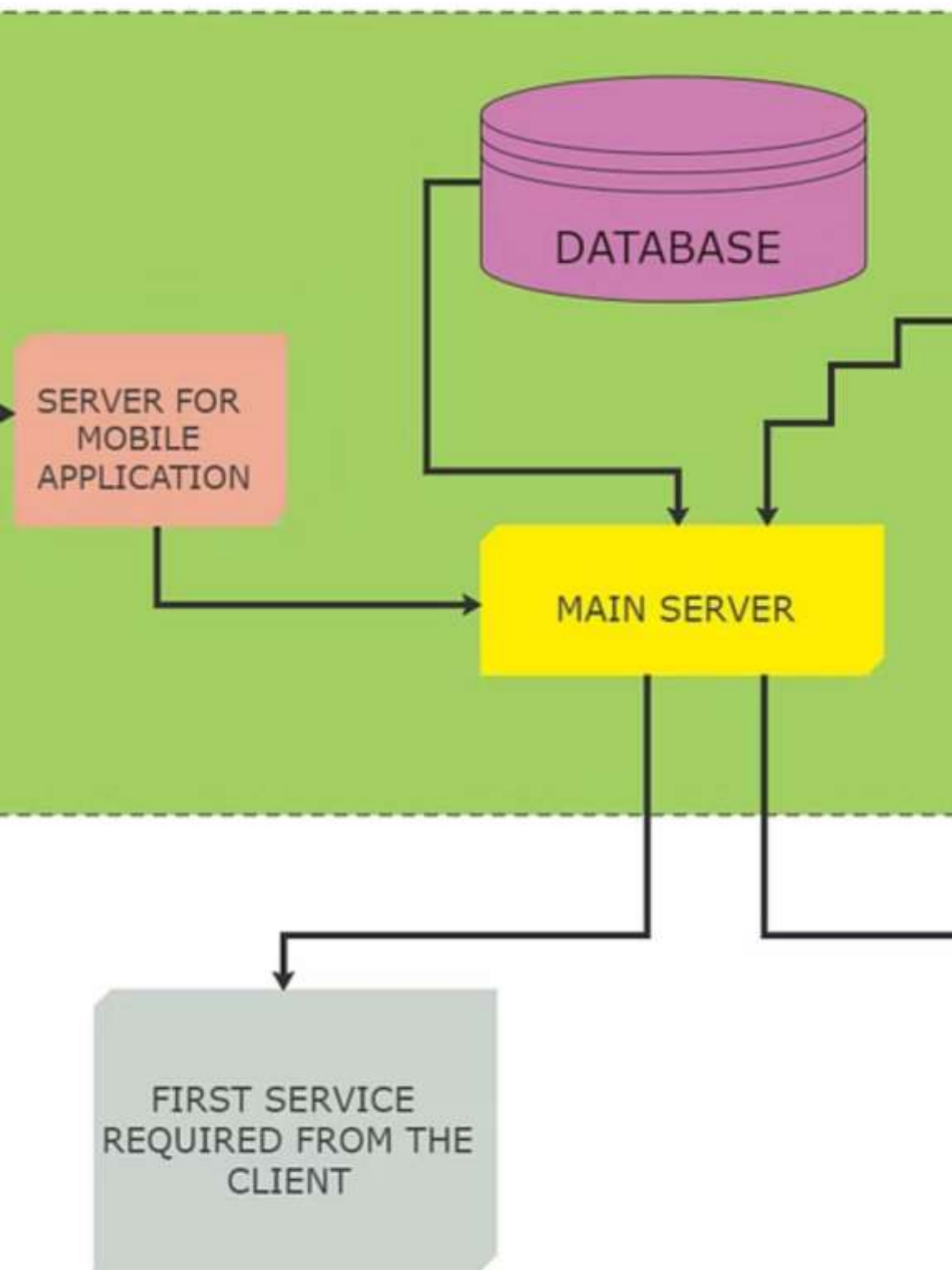
Contoh Sistem Keuangan:

- Transfer dana membutuhkan approval terpisah
- Audit trail independent
- Pemisahan role developer dan operator

Contoh DevOps:

- Code review sebelum merge
- Approval untuk production deployment
- Monitoring oleh tim terpisah

SOFTWARE ARCHITECTURAL DIAGRAM



Prinsip 5: Economy of Mechanism

Kesederhanaan dalam Desain Keamanan

"Keamanan yang kompleks adalah keamanan yang rapuh"

Prinsip Utama:

- Gunakan algoritma dan protokol yang sudah terbukti
- Hindari implementasi custom untuk fungsi kritis
- Minimalisir attack surface
- Dokumentasi yang jelas dan mudah dipahami

Contoh Praktis:

- Gunakan library kriptografi resmi (OpenSSL, Bouncy Castle)
- Implementasi framework security yang mature
- Avoid "rolling your own crypto"
- Single Sign-On daripada multiple authentication

Prinsip 6: Complete Mediation

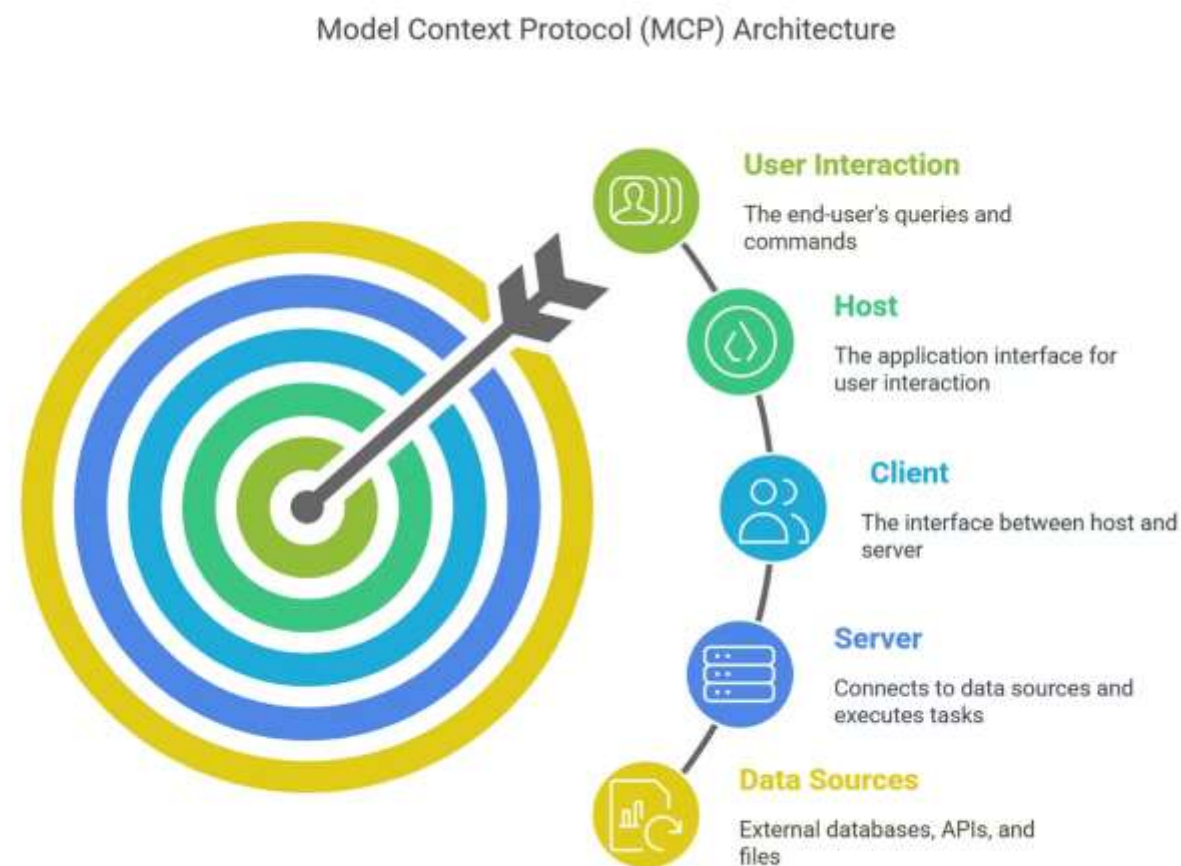
Verifikasi Setiap Permintaan Akses

01	02
Request Received	Authorization Check
Setiap permintaan akses diterima sistem	Validasi kredensial dan hak akses pengguna
03	04
Resource Validation	Access Granted/Denied
Verifikasi ketersediaan dan integritas resource	Keputusan akhir berdasarkan policy yang berlaku

📌 **Contoh Implementasi:** Sistem API yang memvalidasi token JWT pada setiap request, sistem file download yang memeriksa permission user setiap kali file diakses, database yang menerapkan row-level security.

Prinsip 7: Open Design

Keamanan Tidak Bergantung pada Kerahasiaan Algoritma



Security Through Transparency

Kekuatan keamanan sistem harus terletak pada **kunci rahasia**, bukan pada kerahasiaan algoritma atau implementasi.

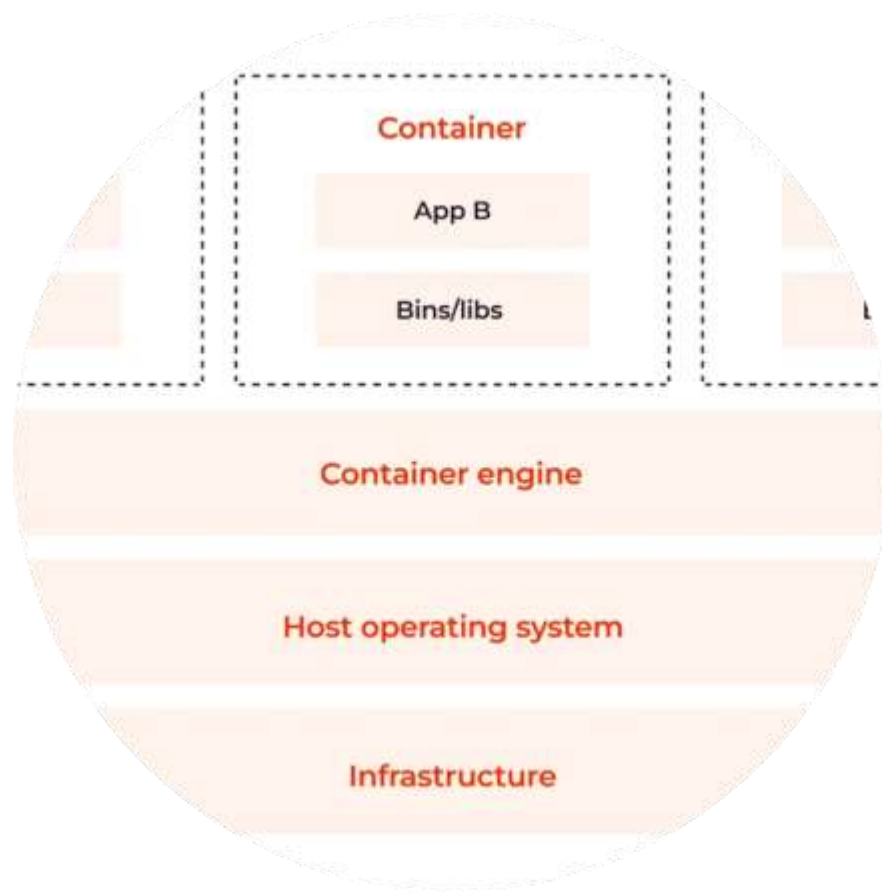
Contoh Protocol Terbuka:

- TLS/SSL: Protokol enkripsi web yang terbuka
- AES: Advanced Encryption Standard
- OAuth 2.0: Framework otentikasi terbuka
- OpenID Connect: Layer identitas di atas OAuth

Algoritma yang telah diuji oleh komunitas global lebih dapat dipercaya daripada "security by obscurity".

Prinsip 8: Least Common Mechanism

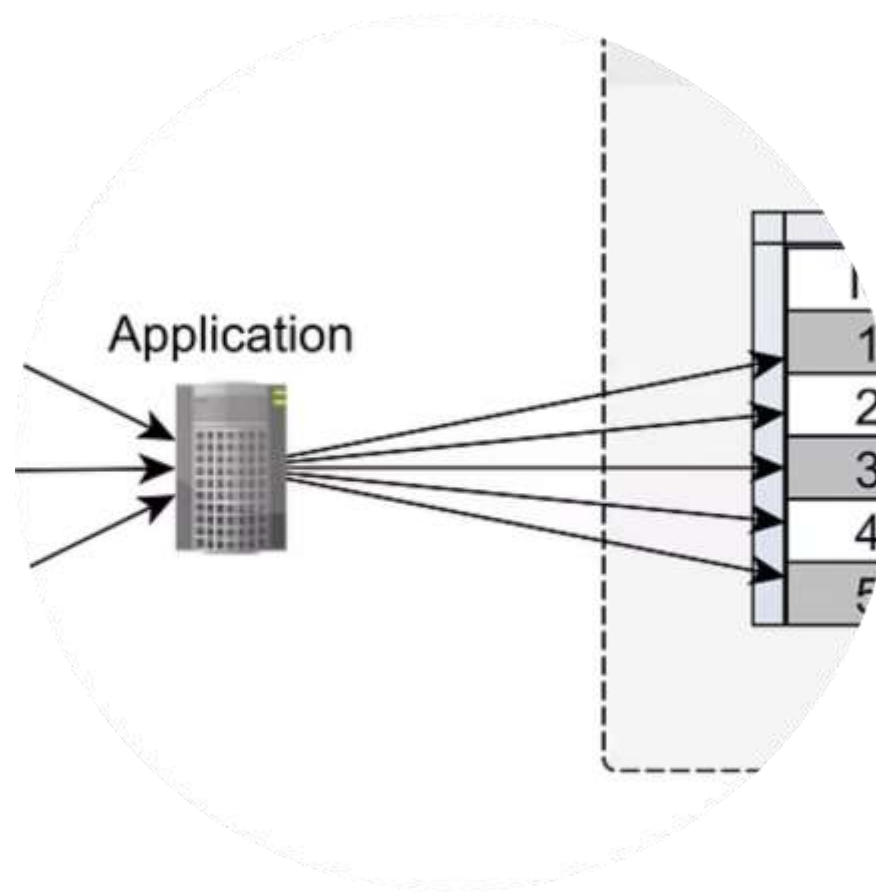
Minimalisir Sumber Daya Bersama



Container Isolation

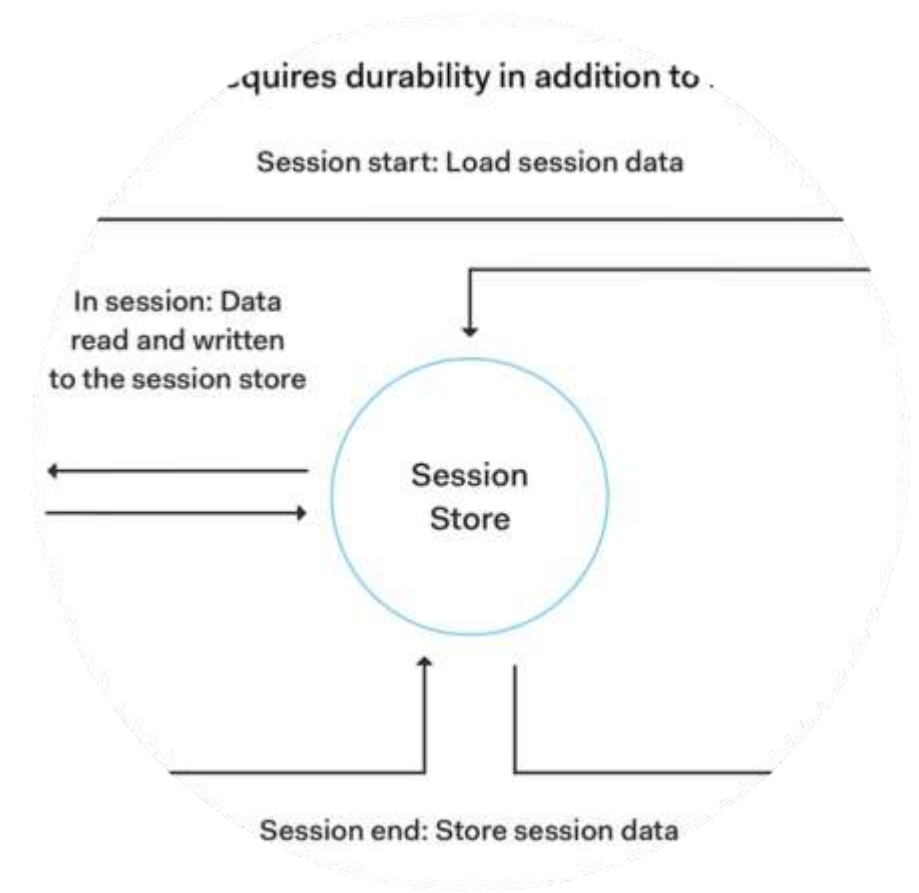
Setiap aplikasi berjalan dalam container terpisah dengan resource yang terisolasi

Pembagian resource dapat menciptakan jalur serangan antar pengguna. Isolasi mengurangi risiko ini.



Database Separation

Multi-tenant architecture dengan schema atau database instance terpisah



Session Management

Session storage dan state management yang terpisah antar pengguna

Prinsip 9: Psychological Acceptability

Keamanan yang Ramah Pengguna

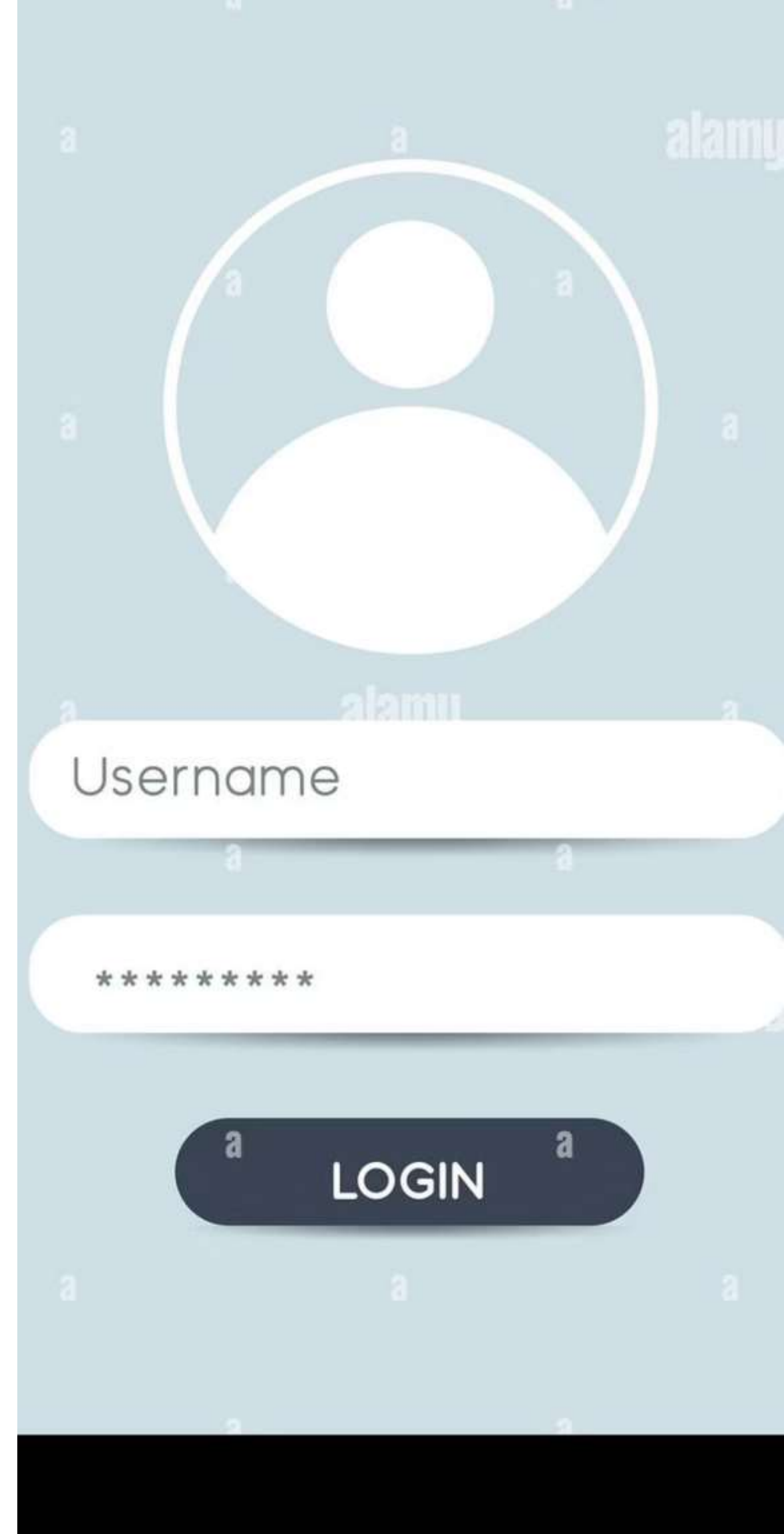
Prinsip Desain UX Security:

- Interface yang intuitif dan mudah dipahami
- Feedback yang jelas saat terjadi error
- Proses keamanan yang tidak mengganggu workflow
- Edukasi pengguna tentang praktik keamanan

Contoh Implementasi:

- Password strength indicator real-time
- 2FA dengan QR code setup yang mudah
- Security notification yang informatif
- Progressive disclosure untuk fitur advanced

❏ Keamanan yang sulit digunakan akan diabaikan atau dibypass oleh pengguna, mengurangi efektivitas keseluruhan sistem.



Studi Kasus: Perbandingan Sistem Login

✗ Desain Tidak Aman

USERNAME AND PASSWORD

WEBSITE or PROGRAM	INTEL	WEBSITE OR PROGRAM	REDDIT
EMAIL ADDRESS	kevinliles@gmail.com	EMAIL ADDRESS	kevinliles@gmail.com
USERNAME	Kevin S. Liles	USERNAME	realtikevin98
PASSWORD	A867scf9	PASSWORD	VW8dHPIQ
ADDITIONAL NOTES		ADDITIONAL NOTES	

WEBSITE or PROGRAM	MICRODREAMS	WEBSITE OR PROGRAM	INSTAGRAM
EMAIL ADDRESS	ksliles@yahoo.com	EMAIL ADDRESS	kevinliles@gmail.com
USERNAME	Kevin S. Liles	USERNAME	realtikevinliles
PASSWORD	79h8t8erts	PASSWORD	kevin1998photos
ADDITIONAL NOTES		ADDITIONAL NOTES	

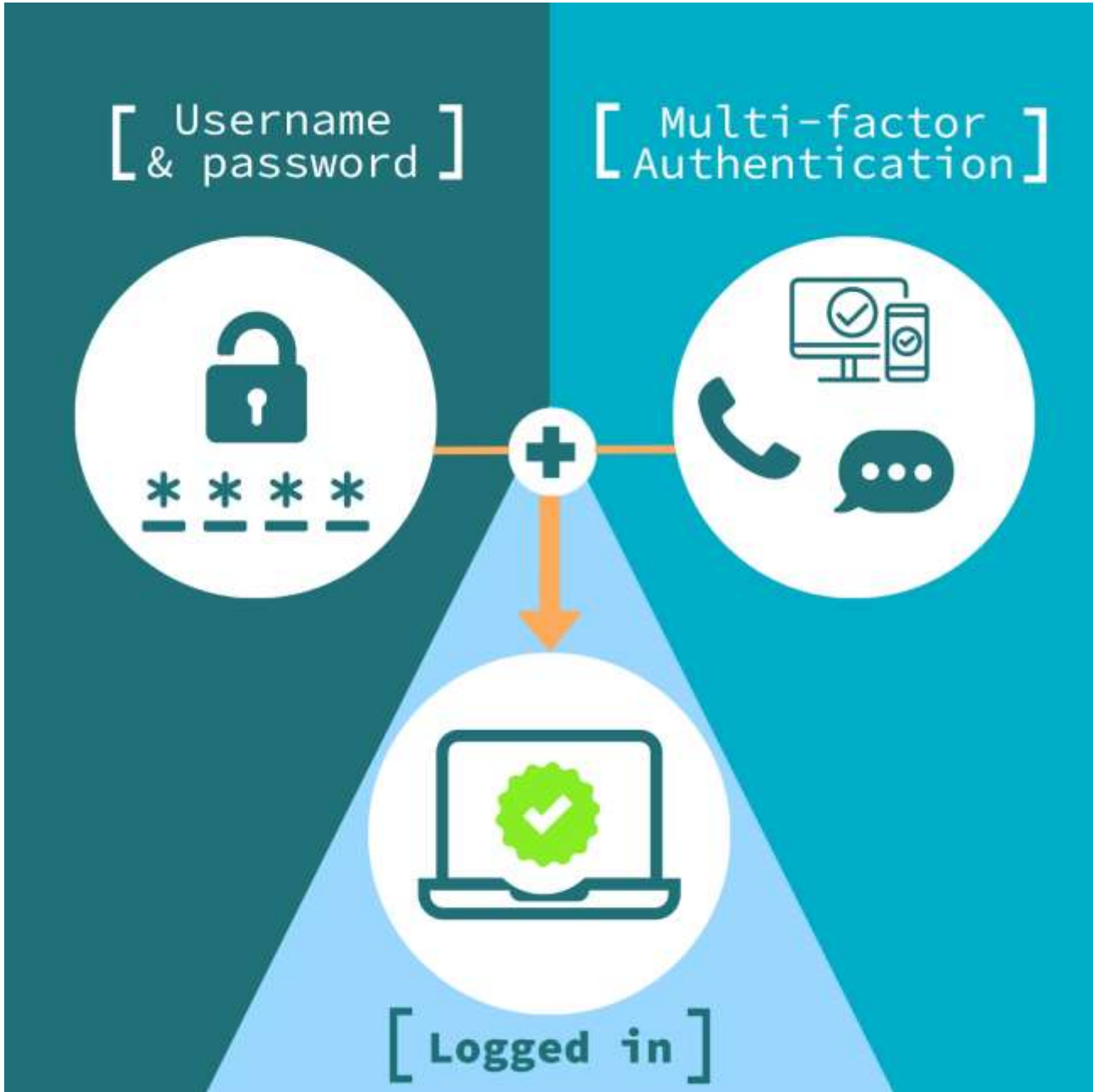
WEBSITE or PROGRAM	REELGEELY FITNESS	WEBSITE OR PROGRAM	ACT ONE COMPUTERS
EMAIL ADDRESS	kevinliles@gmail.com	EMAIL ADDRESS	ksliles@yahoo.com
USERNAME	Kevin S. Liles	USERNAME	Kevin S. Liles
PASSWORD	_a2Hu4X0	PASSWORD	?ABRD,ndd~
ADDITIONAL NOTES		ADDITIONAL NOTES	

WEBSITE or PROGRAM	DISCORD	WEBSITE OR PROGRAM	YAHOO! MAIL
EMAIL ADDRESS	kevinliles@gmail.com	EMAIL ADDRESS	ksliles@yahoo.com
USERNAME	kevinoncall	USERNAME	Kevin S. Liles
PASSWORD	%*mTYeZyPL	PASSWORD	gH*MAo8
ADDITIONAL NOTES		ADDITIONAL NOTES	

- Editable | Printable
- Instant Download

- Password: Disimpan dalam plaintext
- Session: Session ID tidak encrypted
- Error: "Password salah untuk user admin"
- Access: All or nothing privilege

✓ Desain Aman



- Password: Hashed dengan salt (bcrypt/Argon2)
- Session: Encrypted JWT dengan expiration
- Error: "Invalid credentials" (generic)
- Access: Role-based dengan least privilege
- Logging: Comprehensive audit log

Ringkasan: Sembilan Prinsip Desain Aman



Least Privilege

Akses minimum yang diperlukan



Defense in Depth

Perlindungan berlapis



Fail-Safe Defaults

Default ke kondisi aman



Separation of Duties

Pembagian tanggung jawab



Economy of Mechanism

Kesederhanaan desain



Complete Mediation

Verifikasi setiap akses



Open Design

Transparansi algoritma



Least Common Mechanism

Minimalisir sharing resource

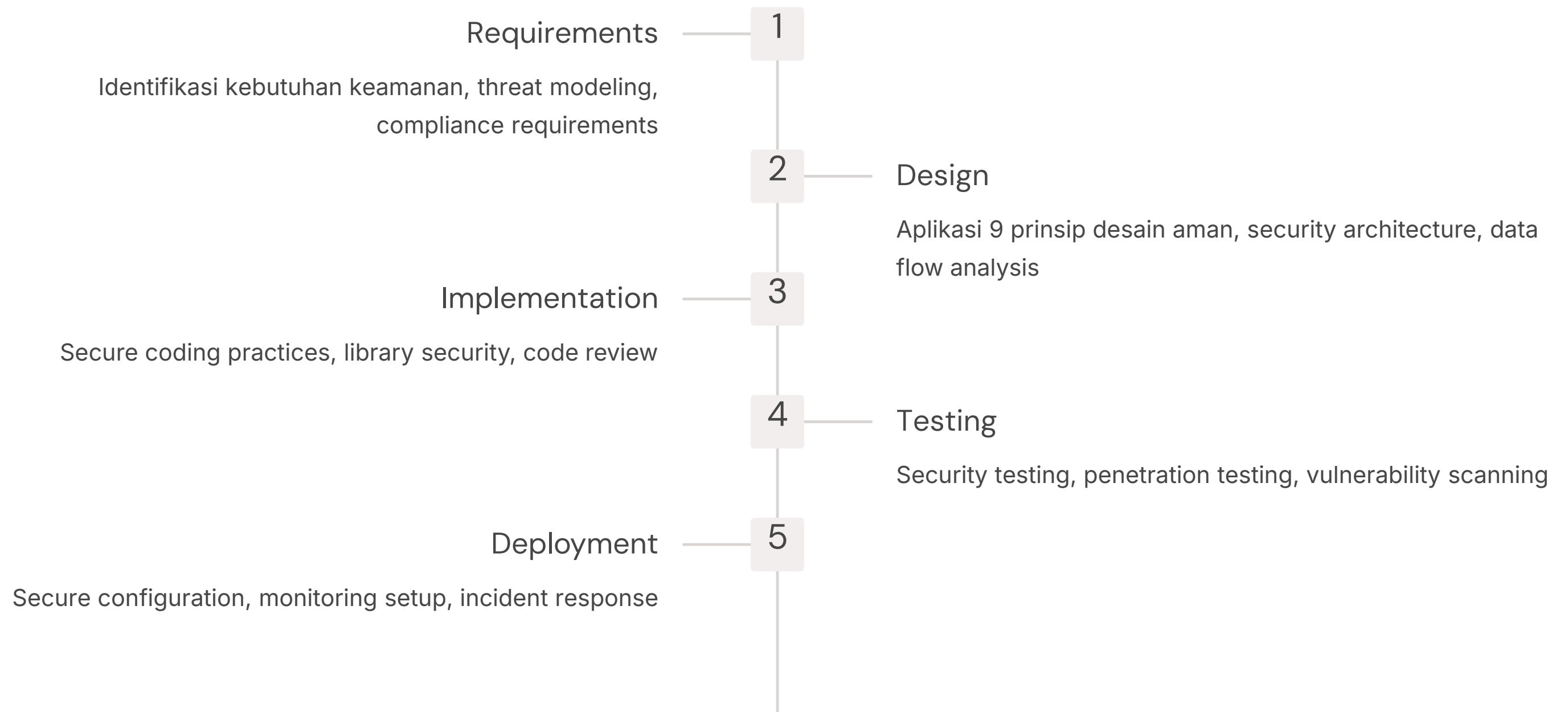


Psychological Acceptability

User-friendly security

Integrasi dengan SDLC

Secure Software Development Life Cycle



Pertanyaan Refleksi

“

Mengapa desain keamanan harus sederhana namun kuat?

Diskusikan hubungan antara kompleksitas sistem dengan potensi kerentanan keamanan. Berikan contoh kasus dimana over-engineering justru mengurangi keamanan sistem.

”

“

Bagaimana prinsip 'least privilege' dan 'defense in depth' saling mendukung?

Analisis skenario dimana kedua prinsip ini bekerja secara sinergis untuk menciptakan sistem yang lebih aman. Berikan contoh implementasi konkret.

”

“

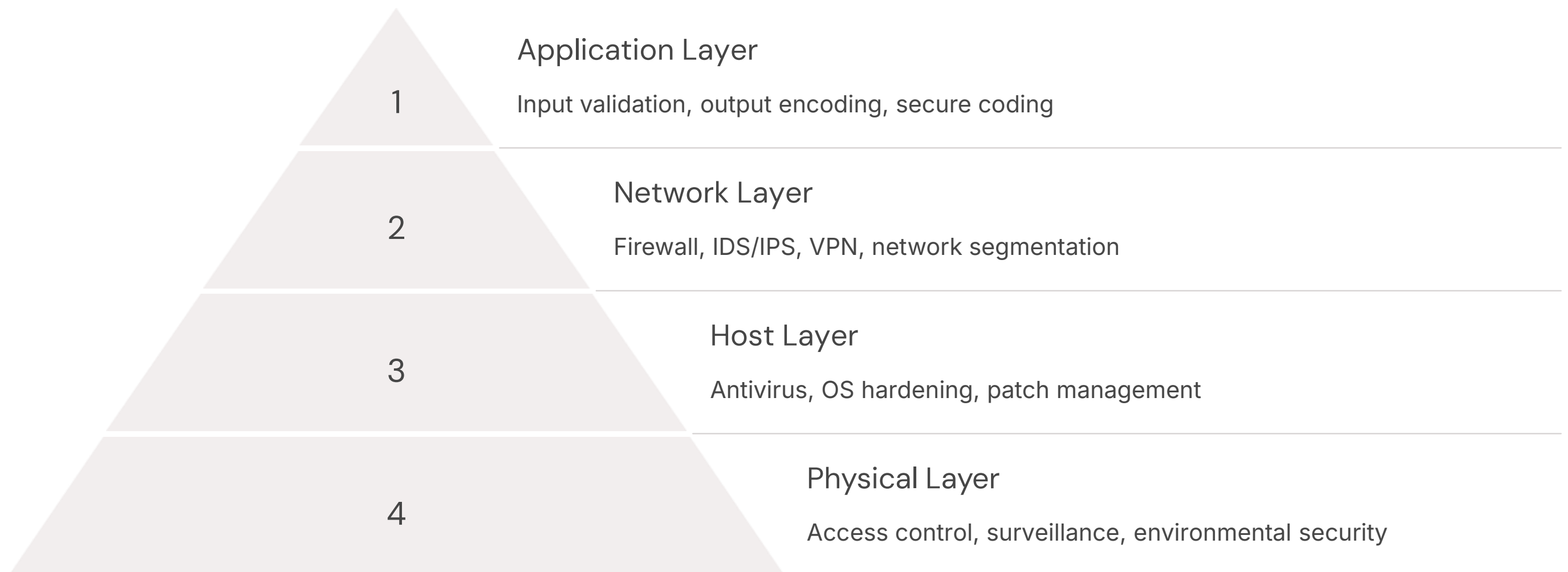
Apa dampaknya jika sistem tidak menerapkan 'fail-safe defaults'?

Evaluasi risiko keamanan yang muncul ketika sistem gagal dalam kondisi yang tidak aman. Diskusikan implikasi terhadap data dan operasional bisnis.

”

- Implementasi least privilege membutuhkan balance antara keamanan dan usability. Over-restriction dapat menghambat produktivitas, sementara under-restriction menciptakan risiko keamanan.

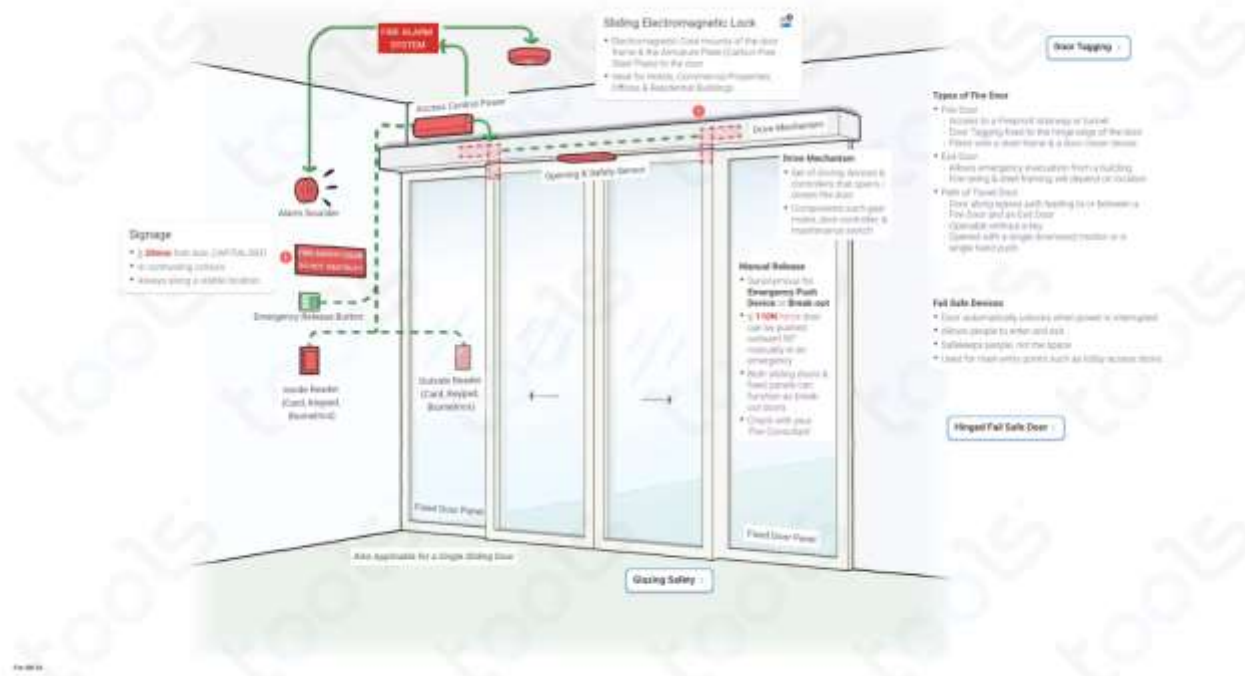
Defense in Depth: Strategi Berlapis



Setiap lapisan memberikan perlindungan independen. Jika satu lapisan dikompromikan, lapisan lain masih dapat mempertahankan integritas sistem secara keseluruhan.

Analisis Risiko: Fail-Safe vs Fail-Open

Fail-Safe (Recommended)



Karakteristik:

- Akses ditolak saat terjadi error
- Sistem masuk ke mode read-only
- Logging tetap berfungsi
- Graceful degradation

Contoh:

Database connection pool penuh → reject new connections
dengan error message yang aman

Fail-Open (Dangerous)

Karakteristik:

- Akses diberikan saat terjadi error
- Bypass security controls
- Potential data exposure
- Compliance violations

Contoh:

Authentication service down → allow all users to access without verification

Separation of Duties dalam DevSecOps



Development

Developer menulis kode dan unit tests, tidak memiliki akses production



Code Review

Senior developer atau architect melakukan review independen



Quality Assurance

QA engineer melakukan testing dengan environment terpisah



Security Review

Security team melakukan vulnerability assessment



Deployment

DevOps engineer dengan privilege khusus untuk production deployment

Setiap tahap melibatkan peran yang berbeda dengan akses dan tanggung jawab yang spesifik, mencegah single point of failure dalam proses development.

Economy of Mechanism: Kesalahan Umum

Custom Cryptography

Membuat algoritma enkripsi sendiri tanpa peer review yang memadai. Lebih baik gunakan library yang sudah teruji seperti libsodium atau Bouncy Castle.

Obscure Configuration

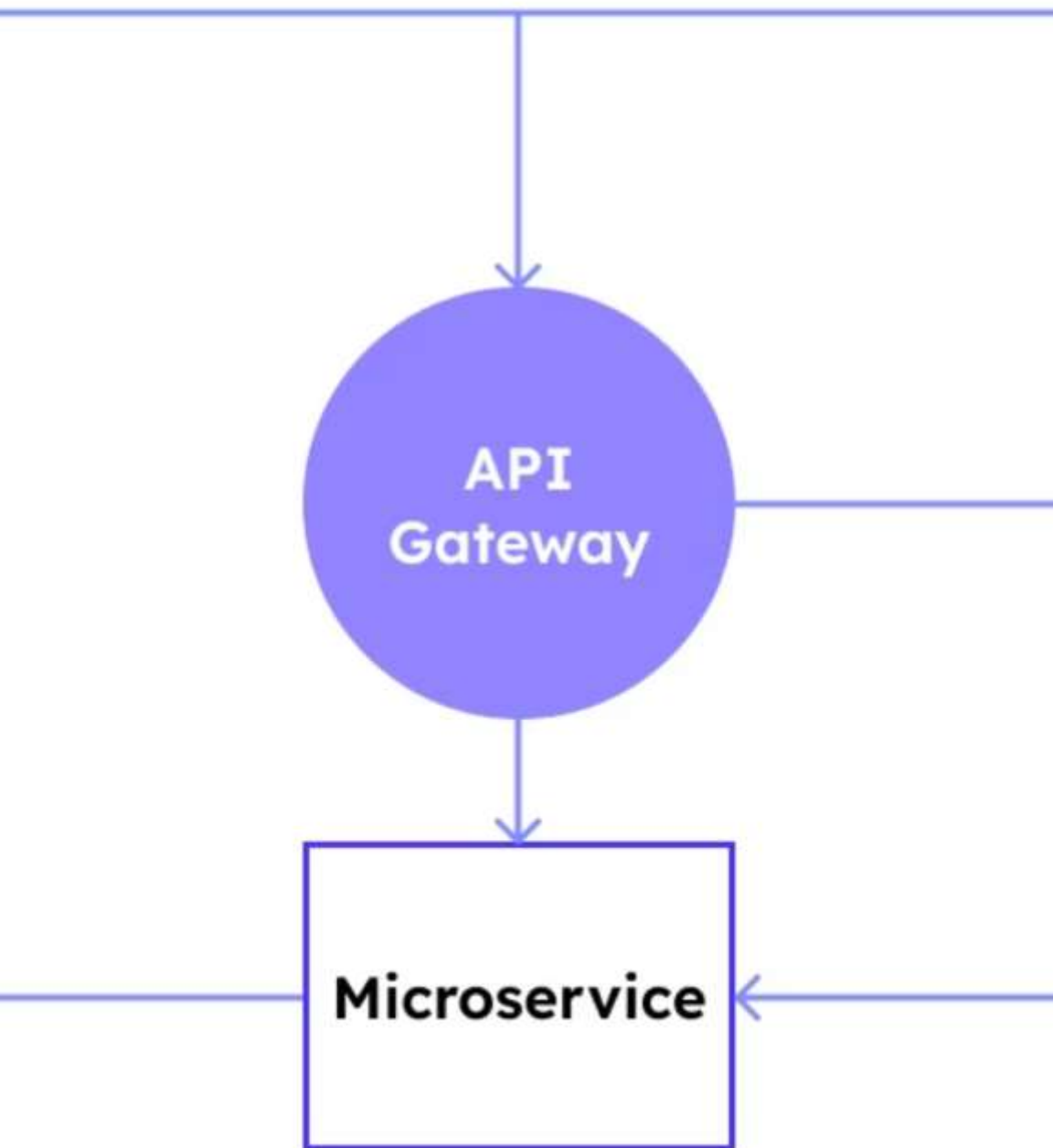
Menggunakan konfigurasi keamanan yang tidak standard atau tidak terdokumentasi dengan baik, menyulitkan troubleshooting dan maintenance.

Over-Engineering Security

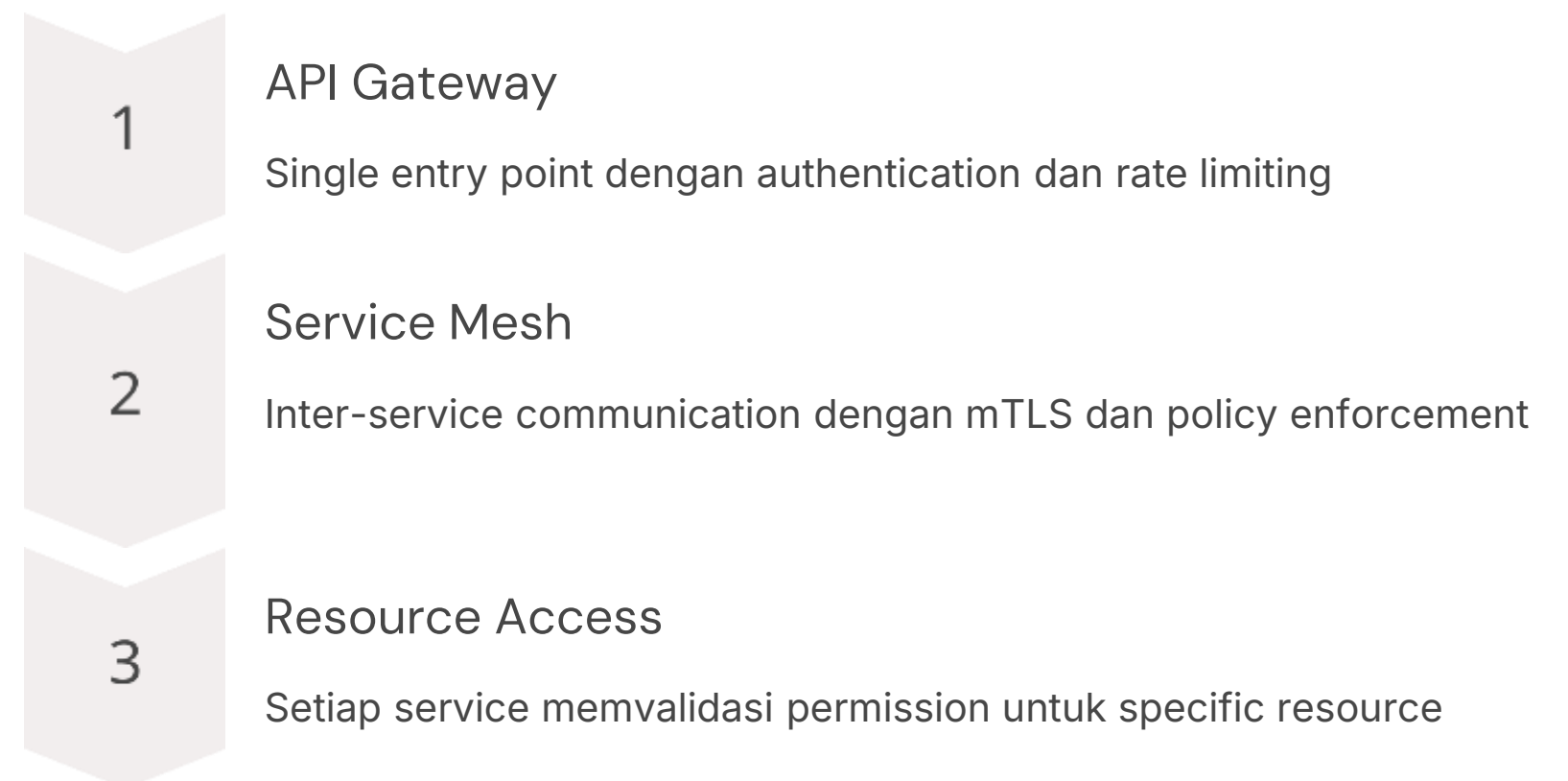
Menambahkan layer keamanan yang berlebihan hingga sistem menjadi sulit dipahami dan dipelihara. Kompleksitas = lebih banyak bug.

Proven Solutions

Gunakan framework dan library yang mature, well-documented, dan memiliki track record keamanan yang baik.



Complete Mediation dalam Microservices



Dalam arsitektur microservices, setiap service harus memvalidasi authorization secara independen, tidak mengandalkan validation dari service lain atau gateway saja.

Open Design vs Security by Obscurity

✓ Open Design Benefits



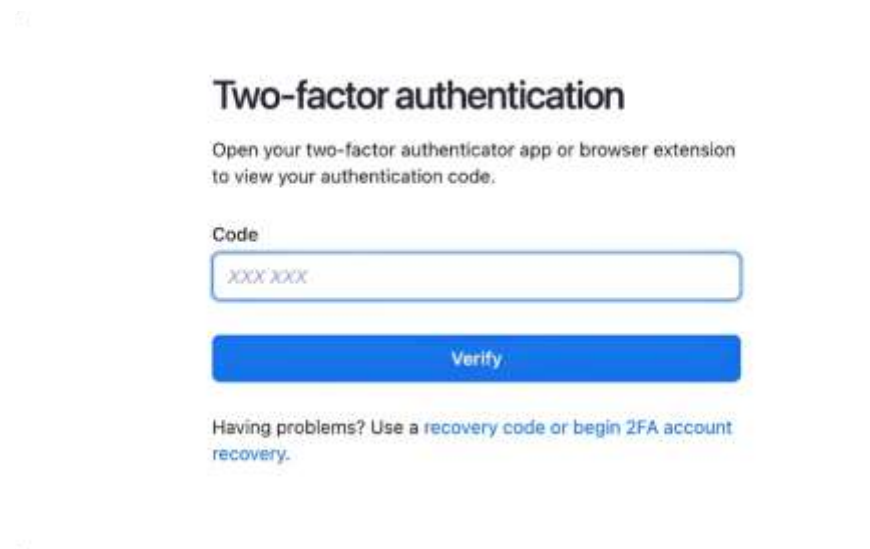
- **Peer Review:** Algoritma diuji oleh ribuan expert
- **Standardization:** Interoperability antar sistem
- **Transparency:** Audit trail yang jelas
- **Continuous Improvement:** Bug fix dan enhancement
- **Trust:** Kredibilitas yang terbangun

✗ Security by Obscurity Risks



- **Single Point of Failure:** Jika algoritma terungkap, keamanan hancur
- **Limited Testing:** Tidak ada external validation
- **Maintenance Burden:** Sulit untuk update dan patch
- **Compliance Issues:** Tidak memenuhi standard industri
- **False Security:** Overconfidence tanpa dasar yang kuat

Psychological Acceptability dalam Desain UX



Password UX

Strength indicator real-time, suggestions yang membantu, progress bar untuk complexity requirements

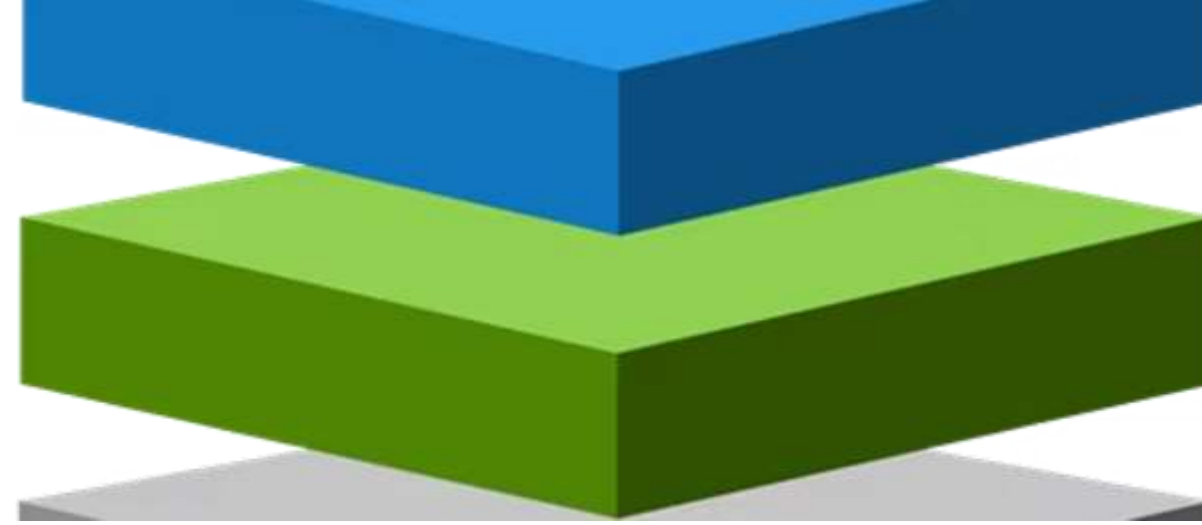
2FA Setup

Wizard step-by-step, QR code yang mudah di-scan, backup codes dengan explanation yang jelas

Security Alerts

Notifikasi yang informatif tanpa menakutkan, actionable steps, konteks yang memadai

- ❏ Keamanan yang sulit digunakan akan di-bypass oleh pengguna. Desain UX yang baik membuat security menjadi invisible dan frictionless bagi pengguna yang legitimate.



Platform-as-a-Service*
(PaaS)

Kesimpulan

Secure by Design

Terapkan Sejak Awal

Integrasikan prinsip-prinsip ini dari fase requirement hingga maintenance

Continuous Learning

Pelajari threat landscape yang terus berkembang dan update knowledge

Praktik Berkelanjutan

Lakukan security review regular dan improvement iteratif pada sistem

Sembilan prinsip desain aman ini bukan hanya teori akademis, tetapi pedoman praktis yang harus diterapkan dalam setiap tahap pengembangan perangkat lunak. Keamanan adalah **investasi jangka panjang**, bukan cost center.